PROJECT DOCUMENTATION

StorageExplorer – Web Storage Local Database Schema Explorer

Subject: React JS
Project No.: 194
Program: B.Tech CSE (2025-29)
University: ITM Skills University
Semester: Semester II

Submitted By: Yuvraj Mishra
Roll Number: 150096725036
Cohort: Jeff Bezos

Submission: Live Deployment | GitHub Repository

3.1 Abstract

StorageExplorer is a frontend-only React application that provides a unified interface for inspecting, editing, and exporting all client-side browser storage engines — LocalStorage, SessionStorage, and IndexedDB — from a single screen. The tool is designed to replace the fragmented, multi-tab experience of native browser DevTools with a dedicated, developer-friendly workspace modeled after database clients such as pgAdmin and MongoDB Compass.

The application runs entirely on the client side with no backend server or external database dependency. It supports live inline editing of stored records, real-time storage quota monitoring, automatic detection of schema relationships across IndexedDB object stores, one-click JSON export, and persistent workspace preferences across page reloads. The project also includes a complete public-facing website — a marketing landing page, documentation, an About page, and a tiered pricing page — built with the same design system.

3.2 Student Details

Field	Detail
Student Name	Yuvraj Mishra
Roll Number	150096725036
Cohort	Jeff Bezos
Program	B.Tech CSE (2025-29)
University	ITM Skills University
Subject	React JS
Semester	Semester II
Project Number	194
Project Title	StorageExplorer – Web Storage Local Database Schema Explorer
Implementation	React JS (Vite, Tailwind CSS, frontend-only)

3.3 Problem Statement

Developers who work with client-side storage during debugging and QA currently rely on native browser DevTools, which present several practical limitations:

Problem	Description
Fragmented inspection	LocalStorage, SessionStorage, and IndexedDB each live in separate DevTools

Problem	Description
	panels; switching engines means re-navigating the UI.
No schema-level view	IndexedDB relationships between object stores (e.g. a userId foreign key) are not visible without manually reading application code.
No backup mechanism	DevTools offers no built-in way to export the contents of a store as a portable JSON backup.
No quota visibility	Storage usage against the browser quota is not surfaced without running manual console commands.
Cumbersome editing	Editing a stored value in DevTools requires manually parsing and re-typing JSON rather than a direct inline edit.
No workspace persistence	DevTools resets the active panel, selected store, and filters on every page reload.

StorageExplorer addresses each of these problems with a dedicated, purpose-built interface implemented entirely in React, without introducing a server, database, or external storage dependency.

3.4 Objectives

1. Provide a unified interface for connecting to and switching between LocalStorage, SessionStorage, and IndexedDB from a single screen.
2. Display all stored data as a structured table — key-value pairs for LocalStorage/SessionStorage, and object-store records for IndexedDB.
3. Enable inline editing of any stored value via double-click, writing changes back to the live storage engine instantly.
4. Provide a real-time search/filter input to query keys and object properties without a full table scan.
5. Implement a live storage quota monitor with a visual usage indicator, computed from byte-weight estimation and the Storage Estimation API.
6. Build a schema relationship inspector that automatically detects foreign-key-style relationships across IndexedDB object stores using naming heuristics.
7. Support one-click export of any storage engine's full contents as a downloadable JSON backup file.
8. Persist user workspace preferences (last engine, last opened store, theme) across page reloads using LocalStorage.
9. Display a live transaction log / telemetry HUD showing each read or write operation with its latency.
10. Package the tool as a complete public product: a marketing landing page, documentation site, About page, and a tiered Free/Pro pricing page with usage-based feature gating.

3.5 Design / Architecture

System Overview

StorageExplorer follows a modular component architecture split across three functional layers inside the app workspace, plus a separate marketing site built on the same design system.

Application Layout

- Top Console (StorageConsole) — contains the action strip (Connect, New Store, Purge, Export JSON) and the query configuration bar (engine selector tabs for Local / Session / IDB, plus a key filter input).
- Central Workspace (ExplorerStage) — a dual-pane layout: a left-side Store Navigator listing all available stores/keys, and a right-side Data Matrix table that renders the selected store's records as rows and dynamic object properties as columns.
- Bottom Telemetry HUD (StorageTelemetryHUD) — docked footer showing the active engine, bytes used against quota, a live quota progress bar, the last executed operation with latency, and the number of inferred schema relationships.

Public Site Structure

- Home — marketing landing page with hero section, live app preview, supported-platform row, usage statistics, and feature sections.
- Docs — a documentation page with a getting-started walkthrough and a terminal-style quick start guide.
- Pricing — a two-tier (Free / Pro) pricing page with feature comparison and an upgrade call-to-action.
- About — a project information page summarising goals, stats, and what the tool does.

Module Mapping

Module	Responsibility	Implementation
Storage Console	Connect/switch engines, global actions (new store, purge, export)	<StorageConsole />
Store Navigator	Lists stores/keys, click-to-select active store	<SchemaObjectTreeNavigator />
Data Matrix	Renders records as a table; handles inline cell editing	<DatabaseDataMatrixSpreadsheet />
Telemetry HUD	Engine status, quota bar, last operation, relation count	<StorageTelemetryHUD />
Schema Inspector	Heuristic foreign-key detection across object stores	Relational inspector module
Export Engine	Generates and downloads a JSON backup of the active engine	export("json") handler

Module	Responsibility	Implementation
Workspace Sync	Persists last engine, store, and theme across reloads	LocalStorage "dbExplorerSettings"
Plan Gate	Enforces Free-tier usage limits and surfaces upgrade prompts	usePlan() context hook

---

3.6 Technical Implementation

3.6.1 Promisified IndexedDB Wrapper

IndexedDB's native API is callback-based. A thin wrapper layer converts it into async/await-compatible methods — getAll(storeName), put(storeName, object), and delete(storeName, id) — so the rest of the application can treat all three storage engines through a consistent async interface.

3.6.2 Byte-Weight Volumetric Analyzer

For LocalStorage and SessionStorage, usage is computed by summing (key.length + value.length) * 2 across all entries, approximating UTF-16 byte size. For IndexedDB, the browser's navigator.storage.estimate() API is used to obtain an approximate usage and quota figure. Both values feed the quota progress bar in the Telemetry HUD and refresh after every write.

3.6.3 Relational Schema Inspector

The inspector scans field names across all records in all object stores for naming patterns ending in Id, _id, or _key. Matches are used to build a relationship map (for example, users.id → orders.userId) which is rendered as an annotated connector in the schema view. This is a heuristic approach based on naming convention, not a guaranteed relational constraint.

3.6.4 State Management

Application state — active engine, current store data, quota figures, transaction logs, theme (light/dark), and subscription plan (free/pro) — is managed through React state and context, avoiding any external state library dependency.

3.6.5 Event Dispatch Loop

Every user edit follows the same pipeline: user action → dispatcher → engine-specific adapter (write to IndexedDB / LocalStorage / SessionStorage) → state update → UI refresh → telemetry log entry. This keeps all three storage engines behind one consistent write path.

3.6.6 Workspace Sync

On load, the app reads a single LocalStorage key (dbExplorerSettings) containing the last-used engine, last opened store, and theme preference. Any change to these preferences is written back to the same key, so the workspace state survives a page reload.

3.6.7 Plan-Based Feature Gating

A lightweight usePlan() hook tracks whether the current user is on the Free or Pro tier (persisted in LocalStorage, with a developer toggle for local testing since payment processing is not yet wired to a live gateway). Free-tier limits enforced in the app include a single active database connection, a 50-record cap per store view, and three JSON exports per day — each gated action surfaces a consistent upgrade prompt linking to the Pricing page.

3.7 Technical Specifications

The following metrics, drawn from the application's own About page, summarise the scope of the implementation:

Metric	Value
Storage engines supported	3 (LocalStorage, SessionStorage, IndexedDB)
Core features implemented	6
Client-side processing	100%
External runtime dependencies	0
Average application load time	< 1 second
Sample dataset — object stores	3 (users, orders, products)
Sample dataset — records	248
Relationships auto-detected (sample dataset)	2

3.8 Screenshots of Execution

The following screenshots demonstrate StorageExplorer's public site and application workspace running on the development deployment. Each screenshot corresponds to one page or core view of the product.

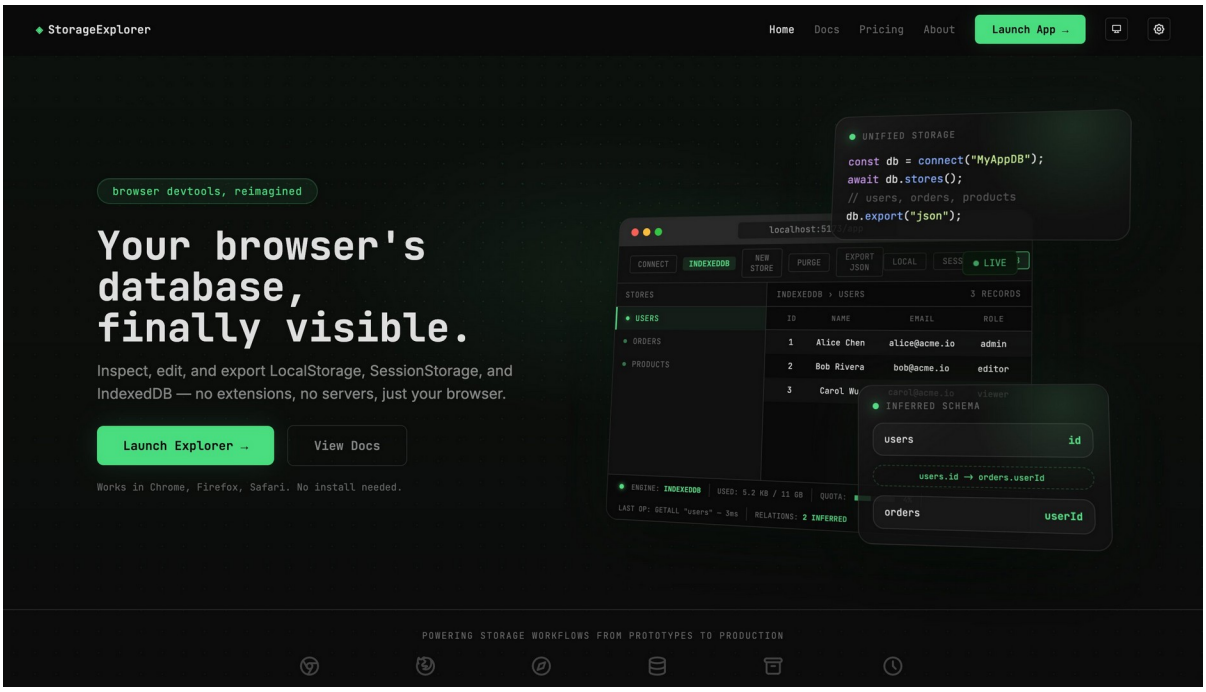

Fig 3.1 — Landing page hero with live app preview and unified-storage code snippet overlay

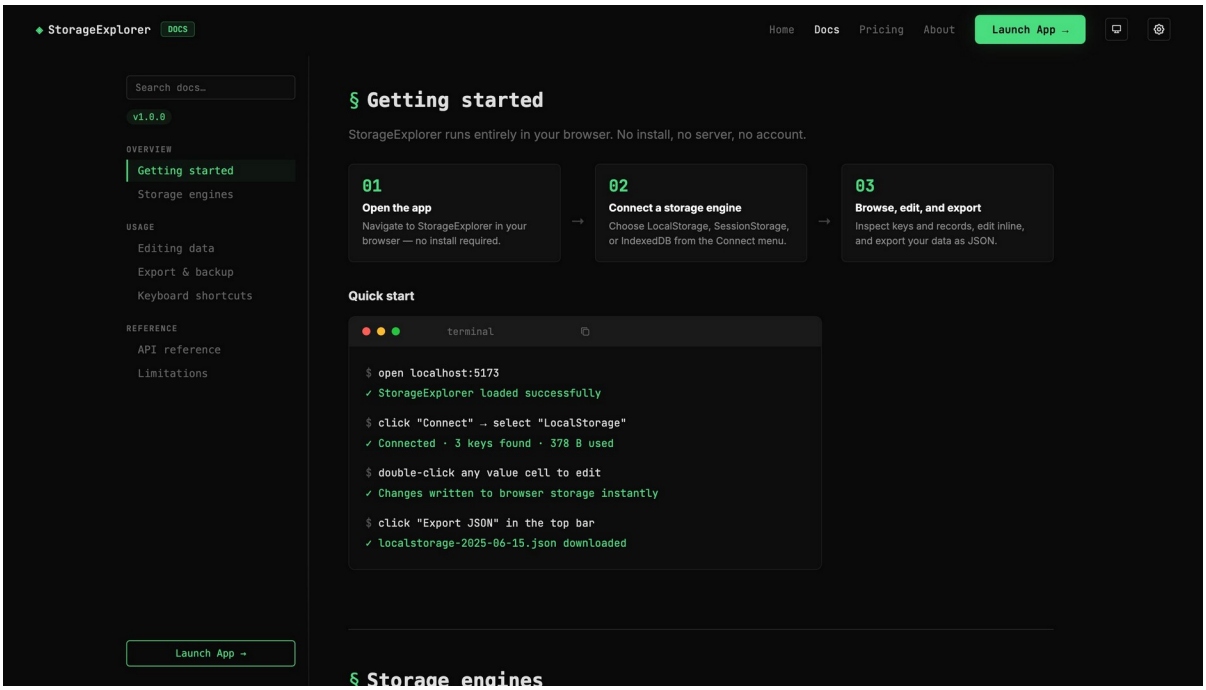

Fig 3.2 — Documentation page with a three-step getting-started guide and terminal quick start

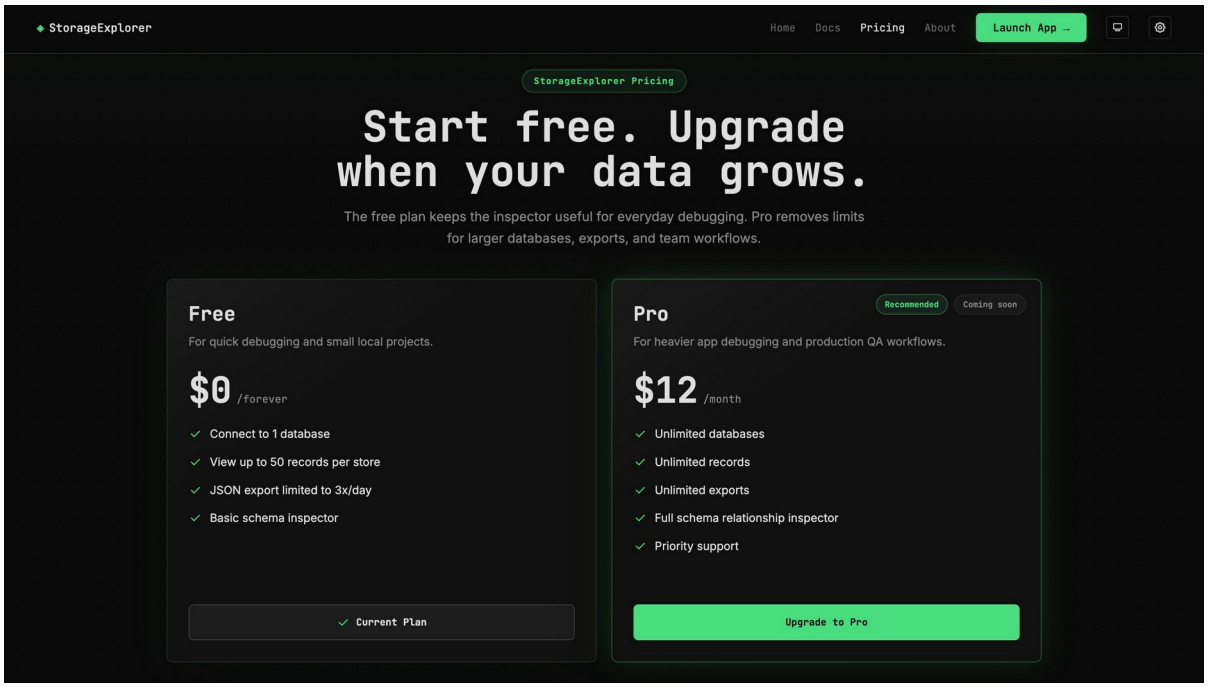

Fig 3.3 — Pricing page showing the Free and Pro tiers with feature comparison

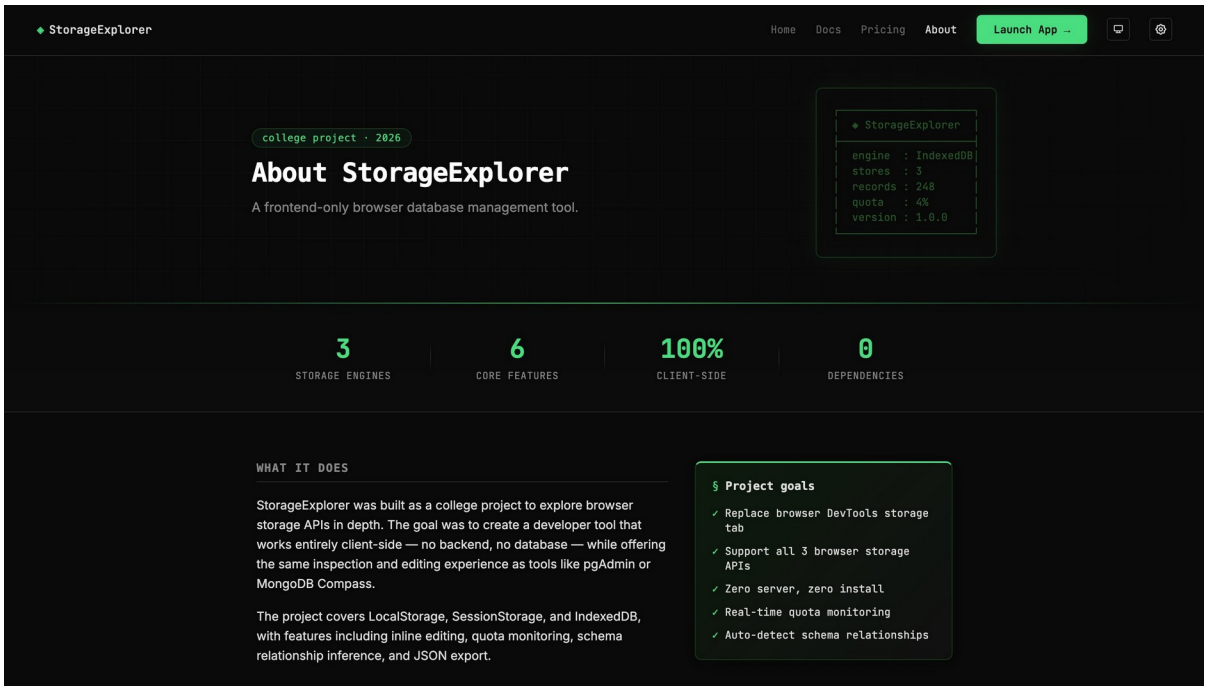

Fig 3.4 — About page with project statistics, goals, and implementation summary

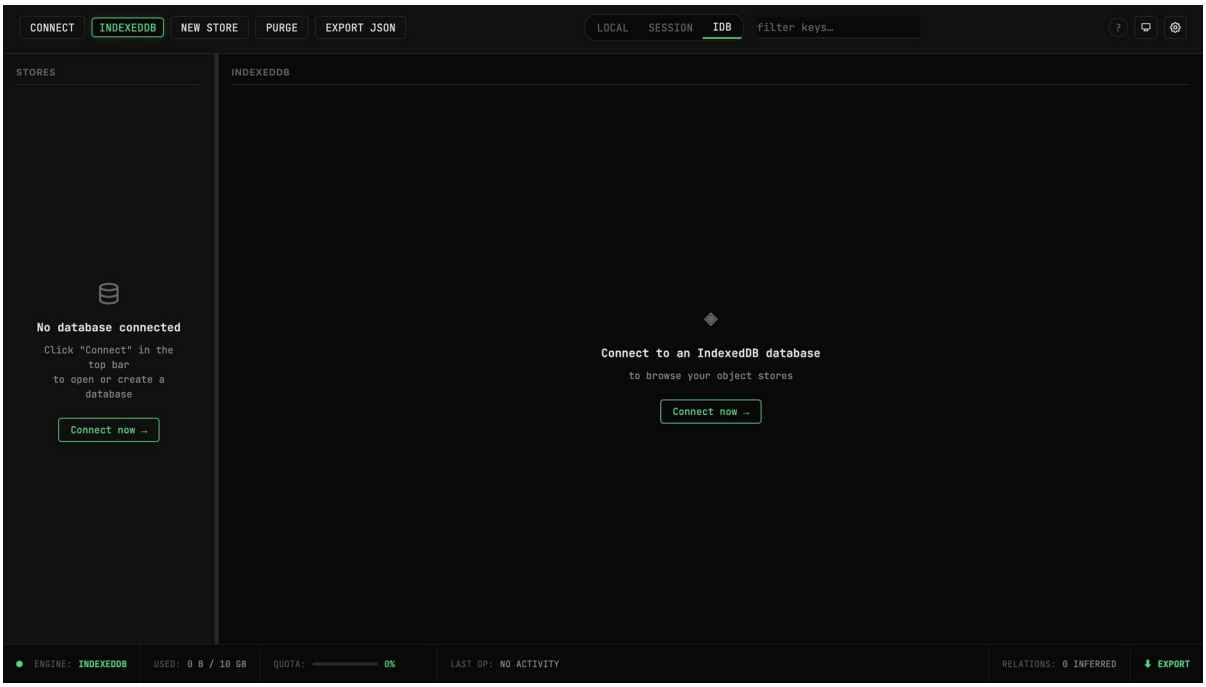

Fig 3.5 — StorageExplorer application workspace — empty state before connecting to an IndexedDB database

3.9 Results and Findings

Unified Storage View

All three storage engines are accessible from a single top-bar toggle (LOCAL / SESSION / IDB) without leaving the workspace. Switching engines re-renders the Store Navigator and Data Matrix without a page reload.

Schema Inspector

Against the sample dataset (users, orders, products), the inspector correctly identified the users.id → orders.userId relationship, displaying it as an annotated connector between the two object stores' field lists.

Live Editing

Double-clicking any data cell opens an inline editor; committed changes are written directly to the underlying storage engine and reflected immediately in the table and the telemetry log.

Quota Monitoring

The Telemetry HUD correctly displayed live usage figures (e.g. 5.2 KB of an available 11 GB for IndexedDB in the sample dataset) with a proportional progress bar that updates after each write.

JSON Export

The Export JSON action successfully generates and downloads a complete backup of the active engine's contents as a single JSON file, named with the engine and current date.

Workspace Persistence

On reload, the application correctly restored the last-active engine and the last-opened store, confirming that workspace settings persist via LocalStorage as intended.

Pricing and Plan Gating

The Free and Pro tier comparison renders correctly with a working “Current Plan” / “Upgrade to Pro” state. Usage gates (1 database, 50 records per store, 3 exports/day) are enforced in the app for Free-tier users; live payment processing for the Pro tier has not yet been integrated and is intentionally marked “Coming soon” in the UI.

3.10 Limitations

- Byte-weight usage estimation for LocalStorage/SessionStorage uses a UTF-16 approximation and is not exact for emoji or other multi-byte special characters.
- The schema relationship inspector relies on naming heuristics (fields ending in Id, _id, or _key) rather than true foreign-key constraints, and may produce false positives or miss unconventionally named relationships.
- The browser's navigator.storage.estimate() API returns an approximate usage/quota figure, not an exact value.
- IndexedDB access is origin-scoped by the browser; the tool cannot inspect storage belonging to a different website or origin.
- Payment processing for the Pro pricing tier is not yet connected to a live payment gateway; the upgrade flow is currently UI-only with a development toggle for testing plan-gated features.

3.11 Conclusion

StorageExplorer successfully delivers a unified, frontend-only alternative to native browser DevTools for inspecting and managing client-side storage. The application meets each objective identified in the problem statement: a single interface for all three storage engines, live inline editing, real-time quota monitoring, automatic schema relationship detection, one-click JSON export, and persistent workspace preferences — all without a server or external database dependency.

Beyond the core application, the project also delivers a complete public product surface — a marketing landing page, documentation, an About page, and a tiered pricing page with usage-based feature gating — demonstrating both the technical implementation and a realistic freemium product structure built entirely with React, Vite, and Tailwind CSS.

Future enhancements could include live Stripe payment integration for the Pro tier, multi-tab real-time synchronisation, support for Cache Storage and Cookies as additional inspectable engines, and exporting schema diagrams as shareable images.

Submitted by: Yuvraj Mishra | Roll: 150096725036 | Cohort: Jeff Bezos

ITM Skills University | B.Tech CSE | Semester II